

INSTRUCTION LEVEL Parallelisme niveau assembleur

LOAD/STORE Architecture

→ Seuls Load/Store

Accèdent la mémoire

R	educed
I	nstruction
S	et
C	omporter

 } $\neq X86$

LOAD source destination

load $\downarrow$ Mem1, $\downarrow$ R₂ (Mem 1) $\rightarrow$ R₂

Store

Store R₅, Mem 2. R₅ $\rightarrow$ (Mem2)

ADD

ADD FP₁, FP₂, FP₆ FP₁ + FP₂ $\rightarrow$ FP₆

Branch

BC2 R₂, (TARGET) IF(R₂) > 0, GO TO TARGET.

I₁

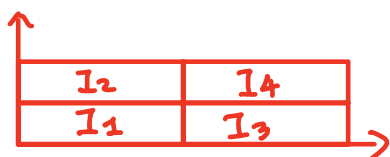
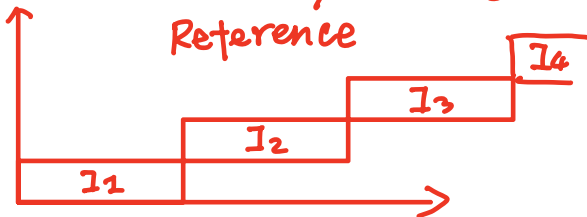
I₂

I₃

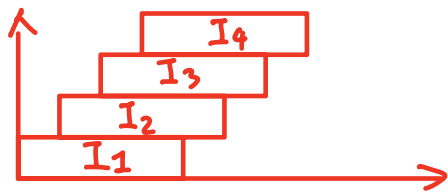
I₄

MODE séquentiel de
Reference

BRANCH UNCONDITIONAL
CA22
TOTU
TOTU
RETURN



Super scalar



	LOAD	STORE	ADD	BC2
IF 2ms	Charger instruction Calcul	"		"
ID 1ms	Changement registre	"		CR
EX 5	Calcul adresse	"	ADD	Calcul target/eval cond
Mem 3	Accès mémoire	"	X	X
WB 2	Ecrit registre destinat°	X	ecrit reg dest	IF(Cond) target → PC

write back

Max : 5ms.

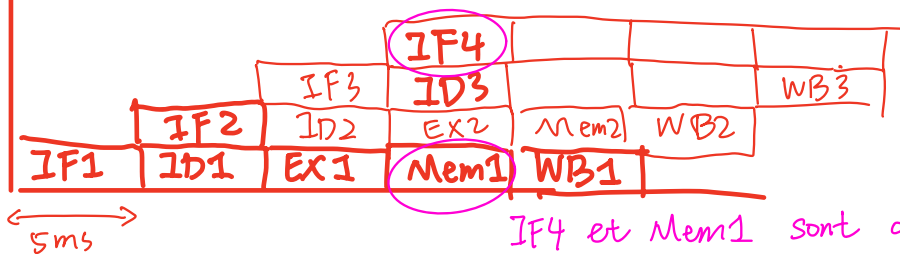
$A > B$?
 $\Updownarrow$
 $A - B$
 $A > 0$

$B \neq 0$

	0	1	2	3	4
IF	I1	I2	I3	I4	I5
ID		I1	I2	I3	I4
EX			I1	I2	I3
Mem				I1	I2
WB					I1

← 5ms →

comportement idéal! (ne pas considérer RAW WAR)



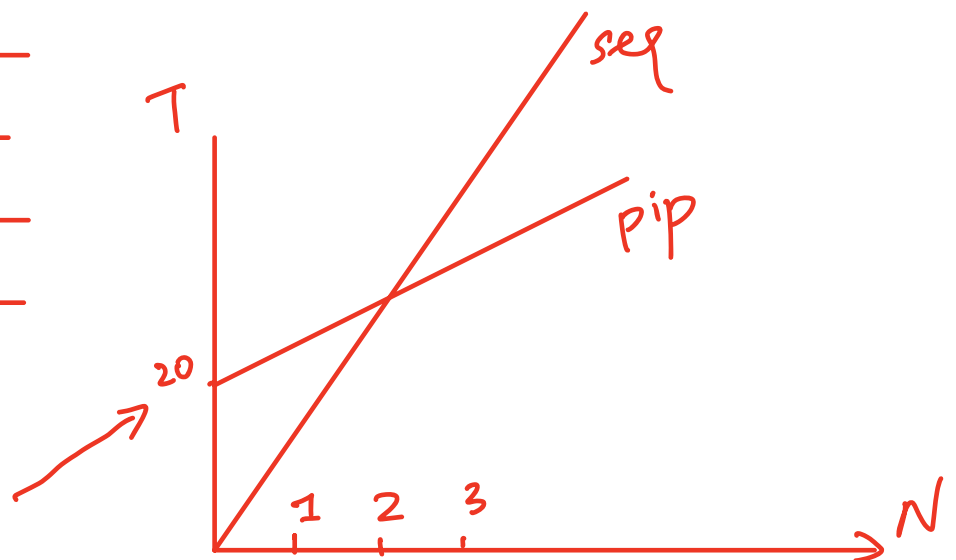
IF4 et Mem1 sont deux accès mémoire

0 1 2 3 4 5 6
 $5 + 5 + 5 + 5 + 5$

	Sequentiel	pipeline
1	13	25
2	26	30
3	39	35

$$T_{seq}(N) = 13 * N \text{ ms}$$

$$T_{pip}(N) = 20 + 5 * N \text{ ms.}$$

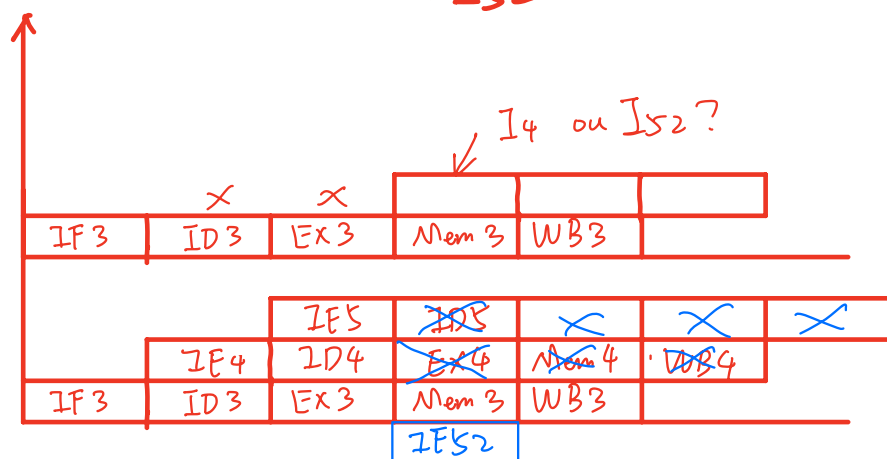


Problème :

→ conflit de ressources (exp. accès de mémoire)

→ BG2 : I3 : BG2 I52

I3 → I4
→ I52



?? 执行了 IF4, ID4, 但不改变状态

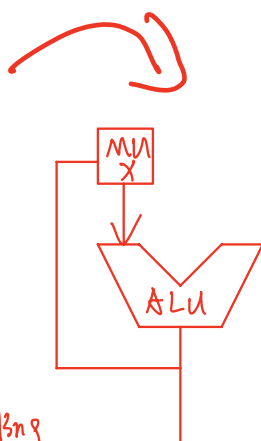
I1 \ I2	R	W
R	RAR	⚠
W	⚠	⚠

Ressource de stockage partage entre I1 et I2

I1 ARITHM → REG

		R2			
R1 + R2 → R3	IF	ID	EX	Mem	WB
R2 * R7 → R6		IF	ID	EX	Mem

	0	1	2	3	4	5
R1 + R2 → R4	IF	ID	EX	Mem	WB	
R4 * R5 → R6		IF	ID	EX	Mem	



Forwarding

	0	1	2	3	4	5
LOAD R4	IF	ID	EX	Mem	WB	
R4 * R5 → R6		IF	ID		EX	Mem

Forwarding

mais perde de cycle

	0	1 $\swarrow R_2$	2	3	4	5
$R_1 + R_2 \rightarrow R_4$	IF	ID	EX	Mem	WB	
$R_7 * R_4 \rightarrow R_2$		IF	ID	EX	Mem	WB

$\swarrow R_2$

	0	1	2	3	4 $\swarrow R_4$	5
$R_1 + R_2 \rightarrow R_4$	IF	ID	EX	Mem	WB	
$R_7 * R_8 \rightarrow R_4$		IF	ID	EX	Mem	WB

$\swarrow R_4$

$\equiv$
 I_1
 $\equiv$
 I_2

Contrainte :

→ : 5 PHASE pour tout le monde.

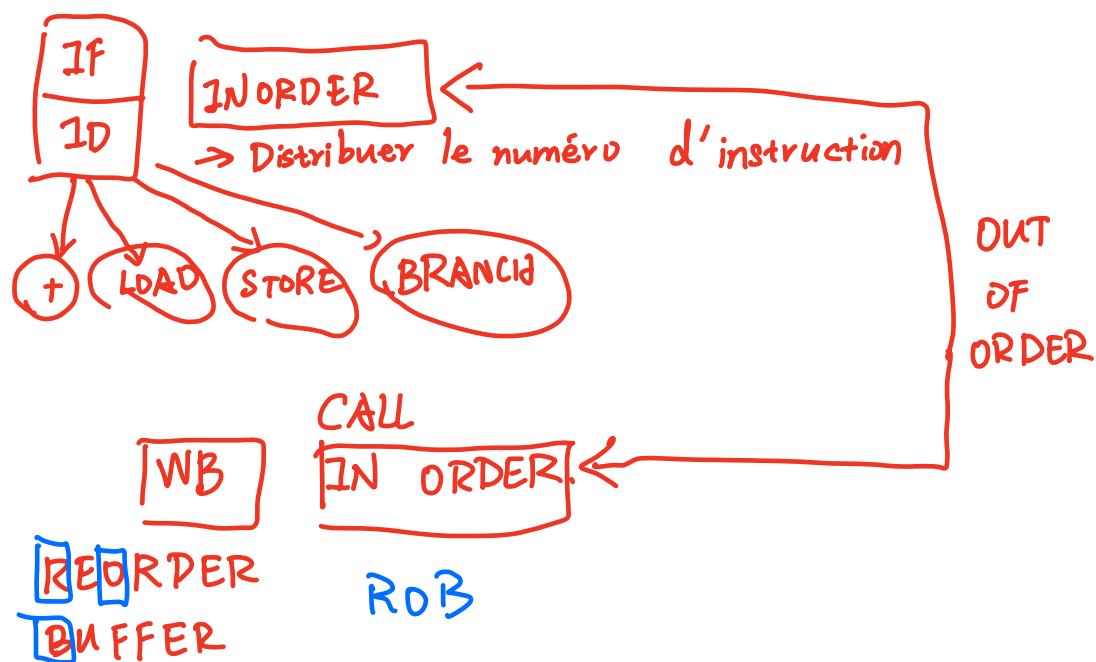
→ 1 Instruction bloquée In order
 toutes les suivant bloquées.

→ tout les étage même durée

IF ID EX
mem WB

Amélioration de pipeline. ① forwarding

② ↓



Complex

CISC.: x86

Instruction.

INST complex.

Set

↓ decoder

Computer

MO P (RISC).

ADD Mem1, R₂, R₃

↓
LOAD Mem1, Tmp → Register.

ADD Tmp, R₂, R₃

RISC : ARM ; Risc V (open source).

DO I = 1, 100,

C[I] = A[I] + B[I]

ENDDO

$A(1:100) \cap C(1:100) = \emptyset$

$B(1:100) \cap C(1,100) = \emptyset$

LOAD @A[I], FP1 A[I] → FP1

INL @A[I]

@A[1] @A[2]

↓
A[1] | A[2] | A[3]

1) code pseudo-code ASM equivalent ?

2) Executer en parallèle ?

3) Executer pipeline ?

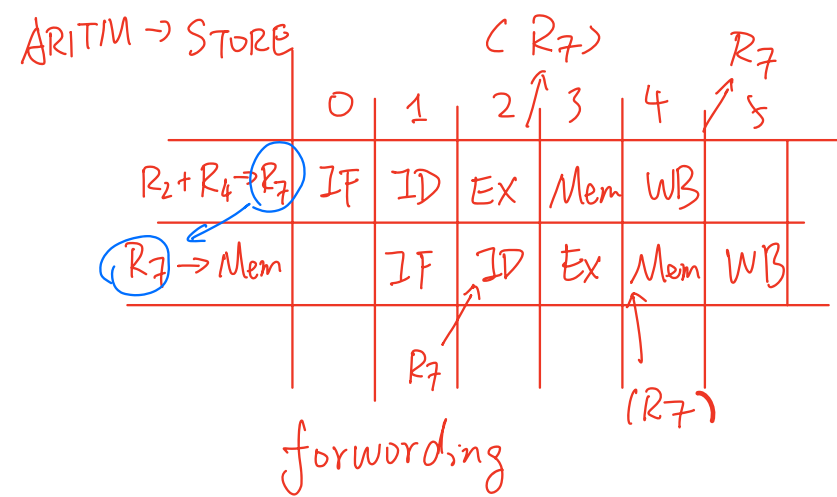
Register renaming

virtuel physical
(16) (64)

XMM0 → PMM3
XMM1 → PMM4 } IT1

XMM0 → PMM5
XMM1 → PMM8 } IT2

BRANCH : PAR1
shadow register ↗ Taken
 ↘ not taken



LOAD Mem 1

↓

LOAD Mem 2

ADRESSE LOAD(R_2) 256, R_7

C(REG) OFFSET $[(R_2) + 256] \rightarrow R_7$

LOAD Mem 1 R_7

STORE R_8 Mem 2

Mem 1 ? = Mem 2

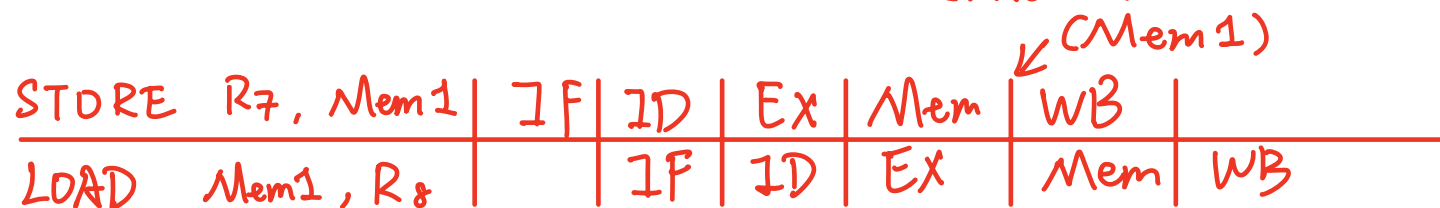
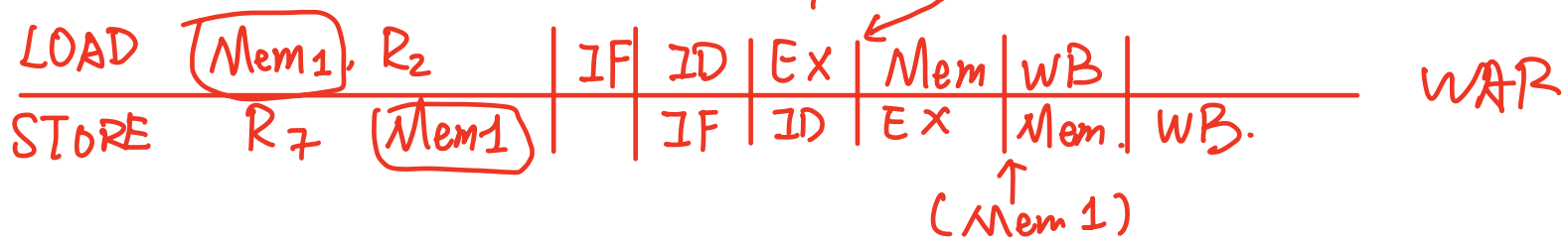
Mem 1 (R_6) 32

Mem 2 (R_8) 12

$(R_6) + 32 \stackrel{?}{=} (R_8) + 12$

Comparaison complete : CHER!

" sur 10 bits Poids faibles (Mem 1)



MOV R_7 , R_8 .

CLASSIFICATION AP

1960: Flynn

DATA STREAM $\nearrow$ SINGLE
 $\searrow$ MULTIPLE

INSTRUCTION STREAM $\nearrow$ SINGLE
 $\searrow$ Multiple

I \ D	S	M
S	SISD	MISD
M	<u>SIMD</u>	<u>MIMD</u>

pas un cas utile exemple: pipeline

DO I = 1, 100 (2,4), 3 STRIDE.
 $C(I) = A(I) + B(I)$

ENDDO

$A(1:100) \cap C(1:100) = \emptyset$

$B(1:100) \cap C(1:200) = \emptyset$

BM_1 BM_2 BM_3 BM_4 bande mémoire.



Réseau interne



unité traitement



unité de contrôle

LOAD A(), R1

LOAD B(), R2

ADD R1, R2, R3

STORE R3, C()

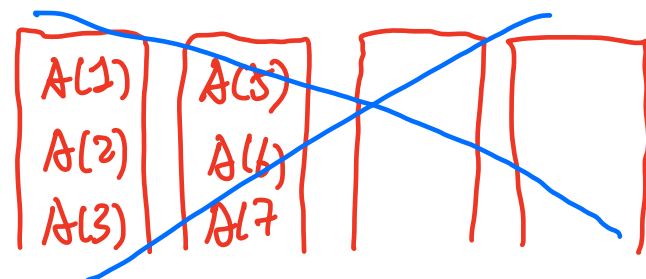
INC @A, @B, @C

100 IT $\rightarrow$ 25 tranches de
 4 iter $\left\{ \begin{array}{l} 1,2,3,4 \rightarrow TR1 \\ 5,6,7,8 \rightarrow TR2 \end{array} \right.$

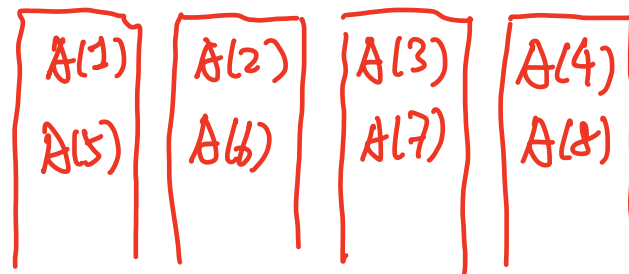
UC	UT0	UT1	UT2	UT3
LOAD	A(1) /	A(2) /	A(3) /	A(4)
LOAD	B(1)	B(2)	B(3)	B(4)
ADD				
STORE	C(1)	C(2)	C(3)	C(4)
INCE	+4	+4	+4	+4
Branch	X	X	X	X
LOAD	A(5)	A(6)	A(7)	A(8)
LOAD	B(5)	B(6)	B(7)	B(8)
ADD				
STORE	C(5)	C(6)	C(7)	C(8)

LOCK
STEP
NODE

TR1



façon parallèle



A(1) → B(1) K/K = 1-1 [4]

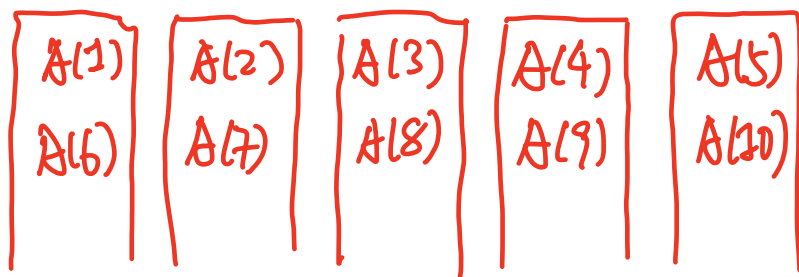
Cyclique

STRIDE $2 \times S \Rightarrow 4 \text{ mots/cycle} \rightarrow 0.5$

$2 \mid S \} \Rightarrow 2 \text{ mots/cycle} \rightarrow 0.25$

$4 \times S$

$4 \mid S \Rightarrow 1 \text{ mots/cycle} \rightarrow 0.25$



A(1) → B(1) K/K = 1-5 [5]

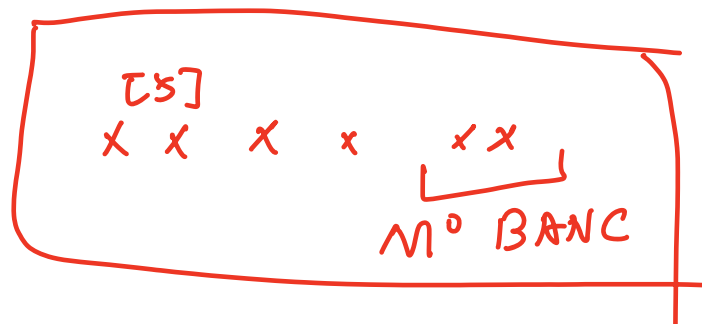
$5 \mid S \Rightarrow 1 \text{ mot/cycle}$ [0.2 | P]

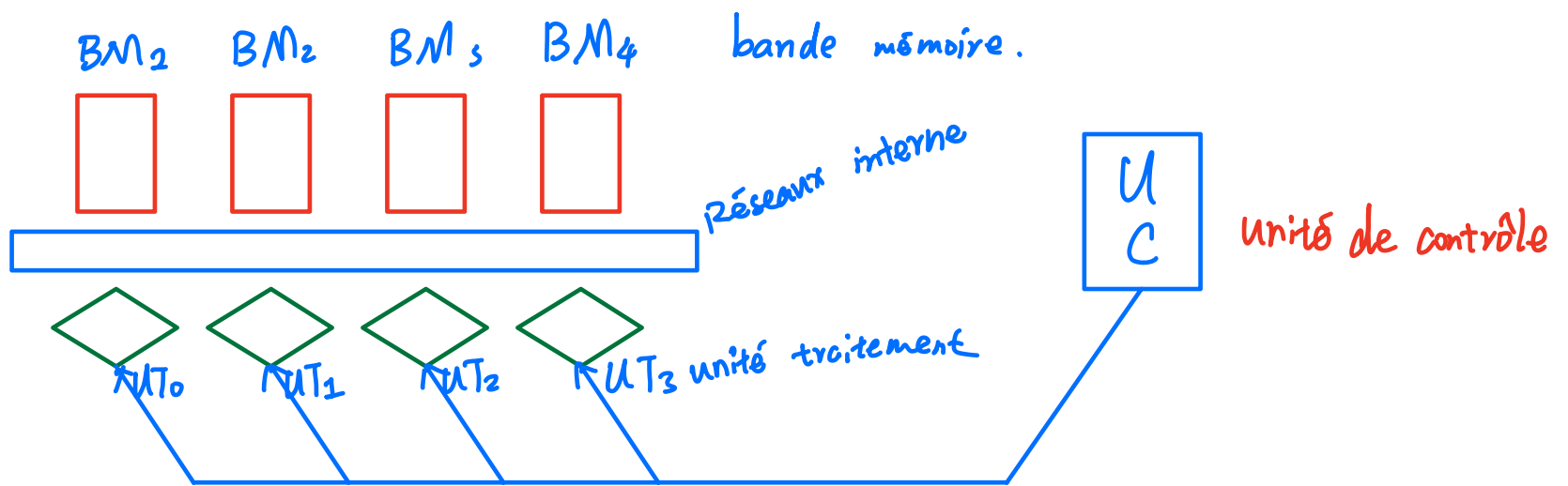
$5 \times S \Rightarrow 5 \text{ mots/cycle}$ [0.8 | P]

$S=1 \Rightarrow P=0.8$

$S=2 \Rightarrow P=0.1$

$S \neq 1, S \neq 2 \Rightarrow P=0.1$





Do $I = 1, 100$

$$C(I) = A(I) + B(I)$$

ENDDO

$$A(I) \rightarrow BM \cdot K /$$

$$K \equiv I - 1 \quad [4]$$

$$A(I, J) \quad \left. \begin{array}{l} 1 \leq I \leq 4 \\ 1 \leq J \leq 4 \end{array} \right\}$$

0	1	2	3
$A(1,1)$	1,2	1,3	1,4
2,1	2,2	2,3	2,4
3,1	3,2	3,3	3,4
4,1	4,2	4,3	4,4

accès par ligne
 D colonne
 O diagonale

$$|i-j| = n. \quad [4] \quad \downarrow$$

0	1	2	3
$A(1,1)$	1,2	1,3	1,4
2,4	2,1	2,2	2,3
3,3	3,4	3,1	3,2
4,2	4,3	4,4	4,1

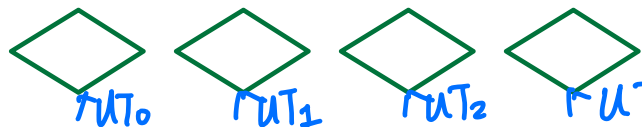
$$A(I, J) \rightarrow BM \cdot K /$$

$$K \equiv I + J - 2 \quad [4]$$

BM₁ BM₂ BM₃ BM₄ bande mémoire.



unité de contrôle



UT₀ UT₁ UT₂ UT₃ unité traitement



DO I = 1, 100

C(I) = A(I) + B(I)

ENDDU

IB = 0
⇒ NO OPERATION

IB = 1

EXEC

STD.

DO I = 1, 100

IF = (A(I) > 0)

THEN T1

ELSE T2

ENDIF

itérat A

(Tranche)

1	>	T ₁
2	>	T ₁
3	>	T ₂
4	<	T ₂
5	<	T ₂
6	<	T ₂
7	>	T ₁
8	>	T ₁

1, 2, 3, 4 → TR 1

5, 6, 7, 8 → TR 2,

↑ TR_1	UC	UT0	UT1	UT2	UT3
	LOAD	A(1)	A(2)	A(3)	A(4)
	CMP	>	>	>	<
	CMP → IB	IB=1	IB=1	IB=1	IB=0
	T_1	T_1	T_1	T_1	
	INV IB	IB=0	IB=0	IB=0	IB=1
↓	T_2				T_2
	Set IB	IB=1	IB=1	IB=1	IB=1
	LOAD	A(5)	A(6)	A(7)	A(8)
	CMP	<	<	>	>
	CMP → IB	IB=0	IB=0	IB=1	IB=1.
	T_1			T_1	T_1
	INV IB	IB=1	IB=1	IB=0	IB=0.
↓ TR_2	T_2	T_2	T_2		

initiation byte.

BGZ $R_1, (52, 12)$

IF $R_1 \geq 0$ (52, 12) → PC

ELSE 458 → PC

CMP2 $R_1 \rightarrow 0001$

COMPARE.

$R_1, R_2, R_3 \Leftrightarrow R_1 - R_2$.
compliquer comme.

Greater Equal.

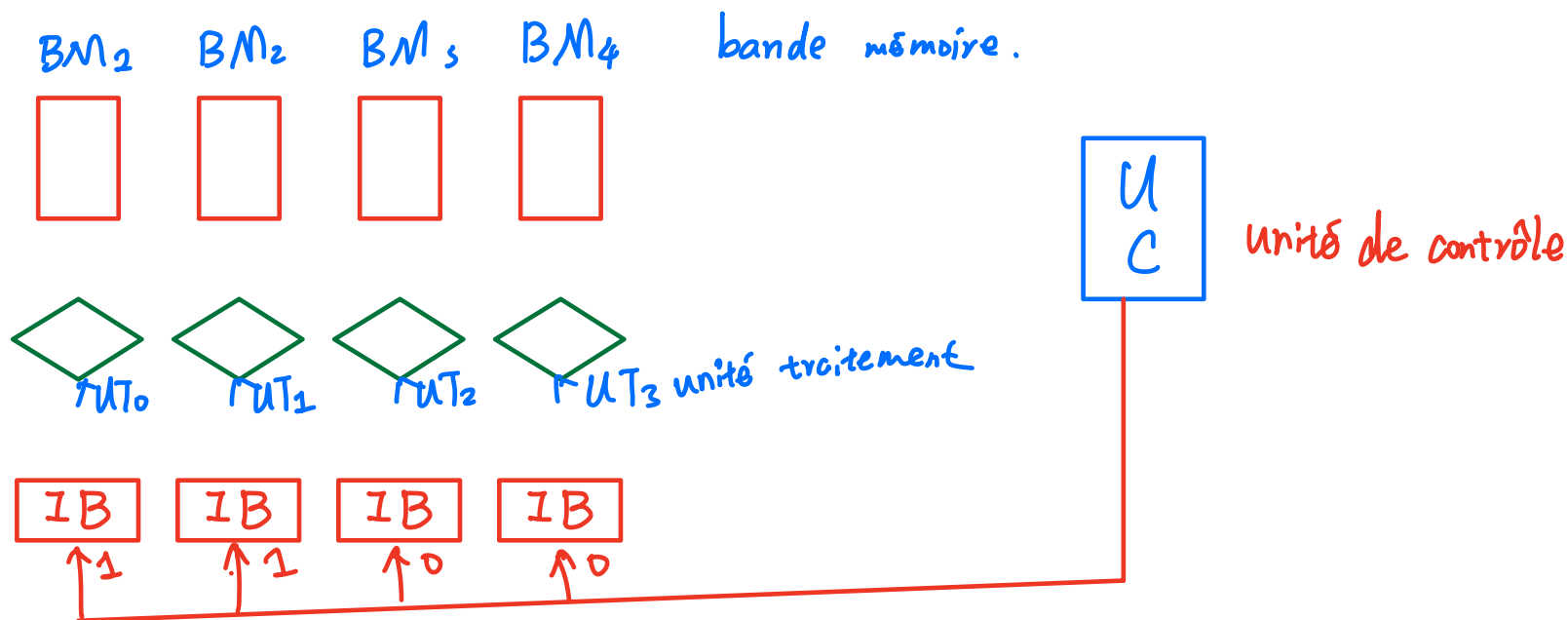
IF $R_1 \geq R_2$ $R_3 = (1, \dots, 1)$

IF $R_1 < R_2$ $R_3 = (0, \dots, 0)$.

>, ≥, <, ≤, =, ≠

=, ≠

Modèle de performance générale



DO $I = 1, N$

$$C(I) = A(I) + B(I) \quad \updownarrow d$$

ENDDU

$T(N)$ division euclidienne

$$N = 4 \times K + R, \quad R \in \{0, 1, 2, 3\}$$

Si $R = 0 \Rightarrow T(N) = K \times d. \Rightarrow$ taux d'utili 25% 50% 75% 100% 62.5 75 87.5 100 ...

Si $R \neq 0 \Rightarrow T(N) = (K+1) d$

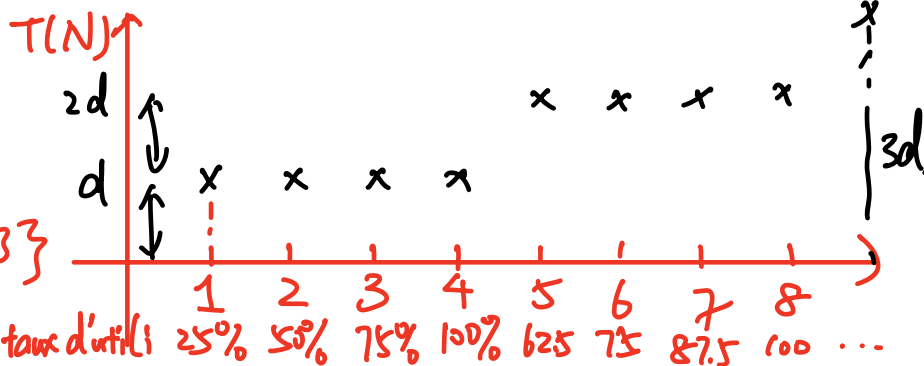
K par rapport à 1

$$K = 1024 \Rightarrow T(4096) = 1024d.$$

$$N = 4096 \Rightarrow 4 \times 1024 = 4096.$$

$$N = 4097 \Rightarrow T(4097) = 1024d + d$$

Temps d'exécution.



$N=1?$

$N=2$

5 $\rightarrow$ TR complete (1, 2, 3, 4)

TR Incomplete (5)

2048 UT

$$T(N) \quad N \leq 2048 \Rightarrow T(N) = d$$

$$T(2049) = 2d$$

(N Versus nombre d'unité traitement)
N versus # UT.

DO $I = 1, 100$

DO $I = 1, N$

IF $= (A(I) > 0)$

THEN T_1 $\boxed{d_1}$

ELSE T_2 $\boxed{d_2}$

ENDIF

$\alpha N \rightarrow T_1$

$(1-\alpha)N \rightarrow T_2$

α : proba que branchement T soit pris.

Temps total d'exécution.

$$T_1(N) = \alpha \cdot N d_1 + (1-\alpha) N d_2.$$

$$= N (\alpha d_1 + (1-\alpha) d_2)$$

SIMD: 4.U.T, $4|N \Rightarrow N=4K$.

$$T_4(N) = K(d_1 + d_2)$$

fait ratio : $\frac{T_1(N)}{T_4(N)} = \boxed{\frac{N}{K}} \cdot \boxed{\frac{(\alpha d_1 + (1-\alpha) d_2)}{d_1 + d_2}}$

Soit $\alpha = 0.5$ $\Downarrow$
 0.5

$$\frac{T_1}{T_4} = 2 \quad 2$$

BM_1 BM_2 BM_3 BM_4 bande mémoire.

$A(1)$
 $A(5)$
 $B(1)$
 $B(5)$

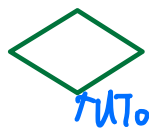
$A(2)$
 $A(6)$
 $B(2)$
 $B(6)$

$A(3)$
 $A(7)$
 $B(3)$
 $B(7)$

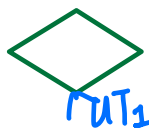
$A(4)$
 $A(8)$
 $B(4)$
 $B(8)$

U
C

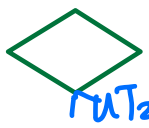
unité de contrôle



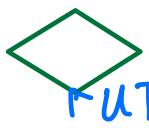
UT₀



UT₁



UT₂



UT₃ unité traitement

IB

$\uparrow 1$

IB

$\uparrow 1$

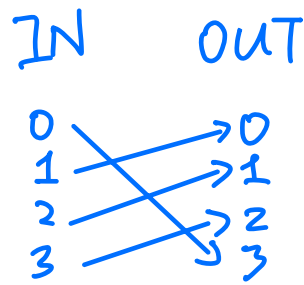
IB

$\uparrow 0$

IB

$\uparrow 0$

TR1
 $A(1) + B(1)$
 $A(2) + B(2)$
 $A(3) + B(3)$
 $A(4) + B(4)$



DO I = 1, 100. +R
 $C(I) = A(I) + B(I+1)$
 ENDDO
 $A(I) \rightarrow B \wedge K / K \equiv I[4]$

TR1
 $A(1) + B(2)$
 $A(2) + B(3)$
 $A(3) + B(4)$
 $A(4) + B(5)$

IN I $\rightarrow$ OUT J.
 $J \equiv I - 1[4]$
 $J \equiv I + P[4]$

- Uniforme.
- parallélisme FP + ACCES
- N nombre itérative.

Versus P
 mbre Ut
 1024 UT
 $N = 1025$
 $N \gg P$

- 1 UC
- Appli // + uniforme
 $\rightarrow$ graphique
 $\rightarrow$ IA (matrice multiplication)

CPU : Unité vectorielle.
 8 à 16 UT

AVX 512 512 bits.

8×64 : DP

16×64 : SP

32×64 : FP16

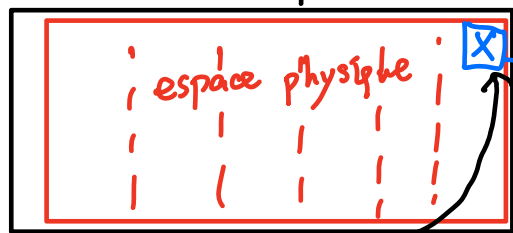
+64b

+32b

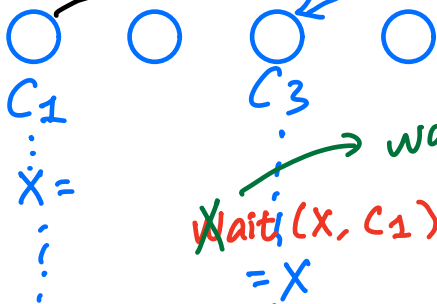
+32b

espace logique (adressage)

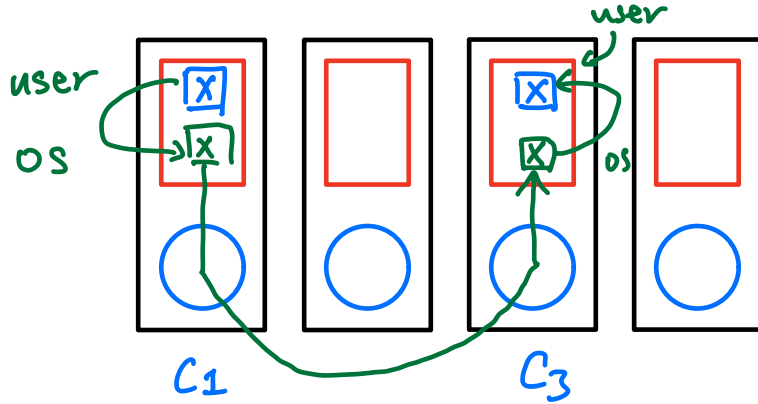
mémoire par tapée
 (shared memory).



réseaux interaction



$\text{wait}(X, C_1) = X$



$X =$
~~SEND~~(X, C3)
 planter
~~RECEIVE~~(X, C3)
 $X =$
 Si pas receive.
 mauvaise valeur
 de mise
**MESSAGE
PASSING
INTERFACE**

DO I = 1, 100

$C(I) = A(I) + B(I)$.

ENDDO

	C1	C2	C3	C4
IT	1-25	26-50	51-75	76-100
A	"	"	"	"
B	"	"	"	"

IT 1-10 11-20 21-30 31-40

Machine à mémoire distribuée.

	C1	C2	C3	C4
IT	1-25	26-50	51-75	76-100
C	"	"	"	"

IT **1-10** 11-20 21-30

send
 (C(11-20), C2)
 send
 (C(21-25), C3)
 Recive
 (C(11-20), C2)
 send
 (C(26-30), C3)
 Recive
 (C(21-25), C1)
 send
 (C(31-40), C4)
 recive
 (C(26-30), C2)

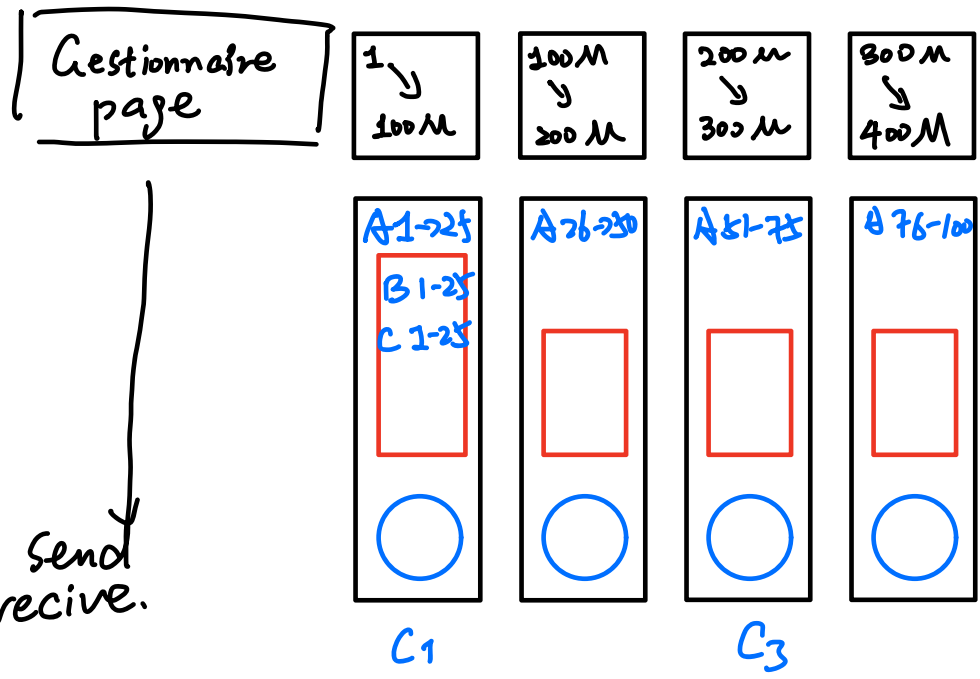
IT 1-25 26-40.
 send
 C(1-25), C3
 C4
 recive
 (C(31-40), C4)

mémoire partagée

- + simple à programmer
- coût CRéseau
- synchronisat°
- performance

mémoire distribuée.

- + coût construire
- programmation. data.
- + perf local
- perf non-local.

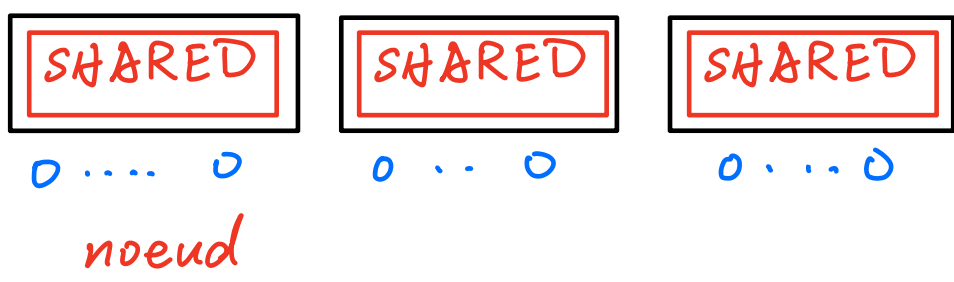


programmation data
mono processeur

disque OS
mémoire virtuelle

Shared virtual memory.

Distributed memory
MPI



10100.